

## Einführung

Ziel dieser Übung ist es, das SIMT-Ausführungsmodell (Single Instruction, Multiple Threads) einer GPU messbar und damit sichtbar zu machen. Dazu habe ich mit PyOpenCL<sup>1</sup> zwei Mikro-Benchmarks gebaut: einen rechenintensiven Kernel, der den Durchsatz in GFLOP/s über die Problemgröße und über den Grad der Warp-Divergenz variiert, und einen speicherintensiven Streaming-Kernel, der die effektive Bandbreite für drei Zugriffsmuster sowie das Verbergen von Latenzen über die Occupancy untersucht. Als Vergleichspunkt dient jeweils eine CPU-Referenz auf Basis von NumPy.

Die OpenCL-Kernels werden zur Laufzeit vom Treiber übersetzt;<sup>2</sup> einzig diese Kernels sind in OpenCL-C geschrieben, die gesamte Steuerung erfolgt in Python. Die hier eingereichten Messungen wurden auf dem OpenCL-Gerät erhoben, das die Bauumgebung bereitstellte, nämlich dem POCL-CPU-Gerät<sup>3</sup> eines AMD Ryzen AI 9 HX PRO 370. Ein dedizierter GPU-Kontext stand in dieser virtualisierten Umgebung nicht zur Verfügung. Der Aufbau ist geräteunabhängig: Mit `python -m gpubench all --device gpu` werden auf der AMD-Radeon-Karte dieselben Abbildungen und Tabellenwerte neu erzeugt. Die folgenden Erklärungen beschreiben daher die hardwareseitigen SIMT-Mechanismen und ordnen die gemessenen Werte ein.

## Aufbau und Messmethodik

Ein `device`-Modul wählt einen OpenCL-Kontext und liest die Geräteeigenschaften aus. Das genutzte Gerät meldet 8 Compute Units, eine maximale Work-Group-Größe von 4096 und ein bevorzugtes Vielfaches der Work-Group-Größe von 8. Dieses Vielfache nutze ich als Stellvertreter für die Wavefront-Breite (auf AMD-GPUs typischerweise 64, bei NVIDIA 32). Die Zeitmessung erfolgt über OpenCL-Profilings-Events, die ausschließlich die Kernel-Laufzeit auf dem Gerät erfassen und damit den Host-Overhead ausklammern. Jede Messung verwendet zwei Aufwärmäufe und meldet anschließend den Median aus mehreren Wiederholungen, um Einschwing- und Ausreißereffekte zu dämpfen. Alle Ergebnisse werden als JSON abgelegt; die Abbildungen entstehen reproduzierbar aus diesen Dateien, sodass der Bericht auch ohne GPU neu gebaut werden kann.

## Aufgabe 1: SIMT und Warp-Divergenz

Im rechenintensiven Kernel bearbeitet jedes Work-Item genau ein Element und führt darauf eine feste Zahl von FMA-Schritten ( $x = x \cdot \text{COEF} + \text{BIAS}$ , zwei FLOPs je Schritt) aus. Abbildung 1 zeigt den Durchsatz über der Problemgröße  $n$ . Bei kleinem  $n$  (1024 Work-Items) liegt der Durchsatz mit 3,6 GFLOP/s niedrig, weil zu wenige Threads vorhanden sind, um die Recheneinheiten auszulasten und Startkosten zu amortisieren. Mit wachsendem  $n$  steigt der Durchsatz und sättigt bei etwa 20,7 GFLOP/s (bei  $n \approx 6,7 \cdot 10^7$ ). Auf einer GPU markiert dieser Sättigungspunkt den Übergang, ab dem alle Compute Units mit genügend Wavefronts gefüllt sind.

Den Kern der Aufgabe bildet die Warp-Divergenz. Im divergenten Kernel wählt jedes Work-Item anhand von `get_local_id(0) % DEGREE` einen von `DEGREE` Pfaden; jeder Pfad

<sup>1</sup>Andreas Klöckner. *PyOpenCL*. 2024. URL: <https://documen.tician.de/pyopencl/>.

<sup>2</sup>Khronos Group. *The OpenCL Specification*. 2024. URL: <https://www.khronos.org/opencl/>.

<sup>3</sup>POCL contributors. *POCL, Portable Computing Language*. 2024. URL: <http://portablecl.org/>.

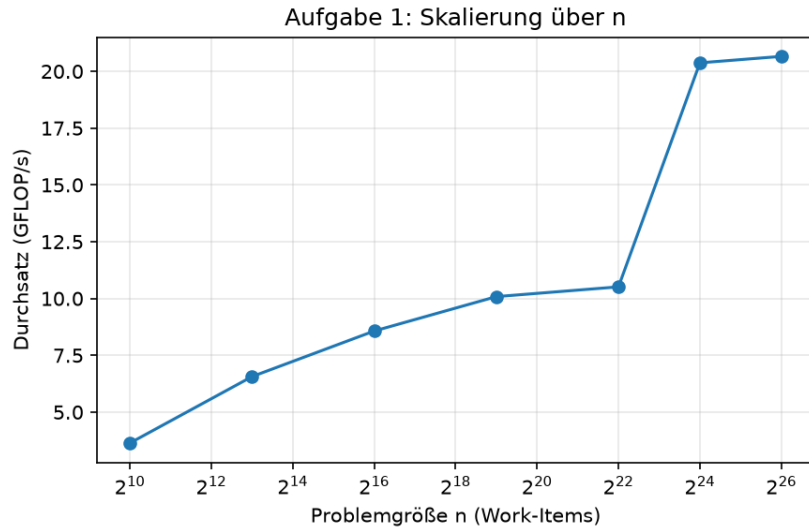


Abbildung 1: Durchsatz über der Problemgröße  $n$  (logarithmische  $n$ -Achse).

leistet exakt dieselbe Arbeit, sodass die Gesamtarbeit je Work-Item unabhängig vom Divergenzgrad konstant bleibt. Auf einer GPU führt eine Wavefront ihre Lanes im Lockstep aus: Nehmen die Lanes unterschiedliche Verzweigungen, so werden die Pfade serialisiert (die jeweils inaktiven Lanes werden maskiert), und der Durchsatz fällt annähernd mit  $1/\text{DEGREE}$ . Abbildung 2 zeigt die Messung auf dem CPU-Gerät: Der Durchsatz sinkt von 20,1 GFLOP/s (kein divergenter Pfad) auf 14,0 GFLOP/s bei Grad 64, also lediglich um den Faktor 1,4. Dieser milde Rückgang ist genau der erwartete Kontrast: Ein CPU-Kern besitzt keine im Lockstep gekoppelte Wavefront, sondern Sprungvorhersage und unabhängige Pfadausführung, sodass Verzweigungen kaum kosten. Die NumPy-Referenz bestätigt dies, ihr Durchsatz bleibt über die Divergenzgrade nahezu konstant. Den charakteristischen steilen  $1/\text{DEGREE}$ -Einbruch liefert erst die Messung auf der GPU.

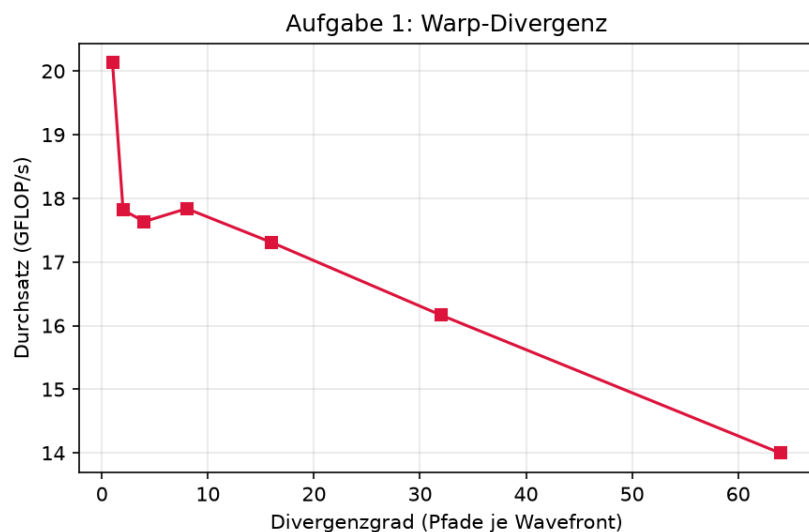


Abbildung 2: Durchsatz über dem Divergenzgrad (Pfade je Wavefront).

## Aufgabe 2: Speicherzugriff und Latency-Hiding

Der speicherintensive Kernel berechnet  $b[i] = a[\text{idx}[i]] \cdot c$  und besitzt eine sehr geringe arithmetische Intensität, ist also bandbreitenbegrenzt. Das Indexfeld `idx` kodiert das Zugriffsmuster, sodass ein einziger Kernel alle drei Muster fair vermisst: coalesced (Identität), strided (großer, zu  $n$  teilerfremder Schritt, hier 521) und gather (zufällige Permutation mit festem Seed). Abbildung 3 zeigt die effektive Bandbreite. Der coalesced-Zugriff erreicht 46,3 GB/s und liegt damit nahe an der separat gemessenen Kopier-Spitzenbandbreite von 44,5 GB/s. Der strided-Zugriff bricht auf 11,6 GB/s ein, der gather-Zugriff auf 9,2 GB/s; coalesced ist somit rund fünfmal schneller als gather. Auf einer GPU verschmilzt die Hardware benachbarte Zugriffe einer Wavefront zu wenigen Speichertransaktionen (Coalescing); unregelmäßige Adressen zerfallen dagegen in viele einzelne Transaktionen und lasten die Bandbreite schlecht aus.

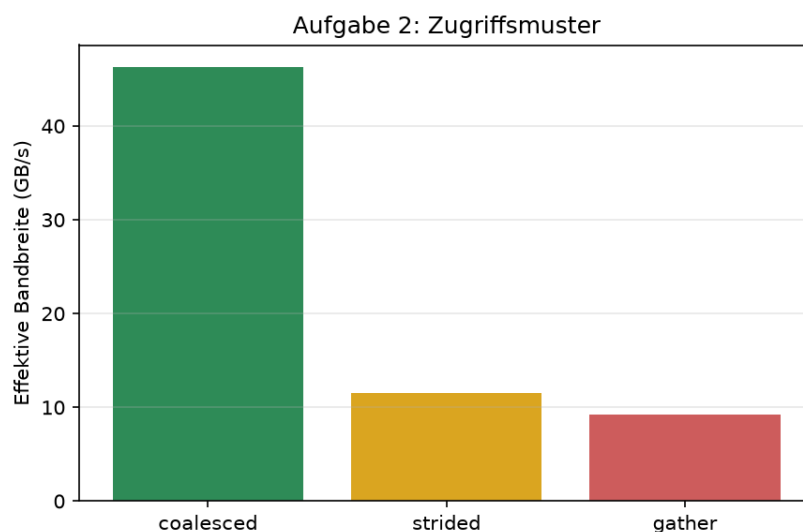


Abbildung 3: Effektive Bandbreite je Zugriffsmuster.

Abbildung 4 variiert die Work-Group-Größe und damit die Occupancy, also das Verhältnis aus aktiven zu maximal möglichen Wavefronts je Compute Unit. Die Bandbreite steigt von 37,4 GB/s bei 8 Work-Items je Gruppe auf 48,5 GB/s bei 512 und läuft dann in ein Plateau. Die GPU verbirgt Speicherlatenz nicht über große Caches, sondern über massive Parallelität: Wartet eine Wavefront auf Daten, schaltet der Scheduler auf eine andere lauffähige Wavefront um. Erst wenn genügend Wavefronts resident sind, wird die Latenz vollständig verdeckt. Die CPU verfolgt die entgegengesetzte Strategie und verbirgt Latenz über die Cache-Hierarchie und Prefetching: Die NumPy-Referenz erreicht sequenziell 5,6 GB/s, beim zufälligen gather aber nur 1,1 GB/s, weil dann nahezu jeder Zugriff einen Cache-Miss erzeugt.

## Fazit

Tabelle 1 fasst die Messungen zusammen. Sie stellt das OpenCL-Gerät (hier POCL-CPU) der NumPy-Referenz gegenüber; auf der AMD-Karte erzeugt derselbe Ablauf die GPU-Spalte.

Beide Experimente zeigen dieselbe Konsequenz des SIMT-Modells: Hoher Durchsatz ent-

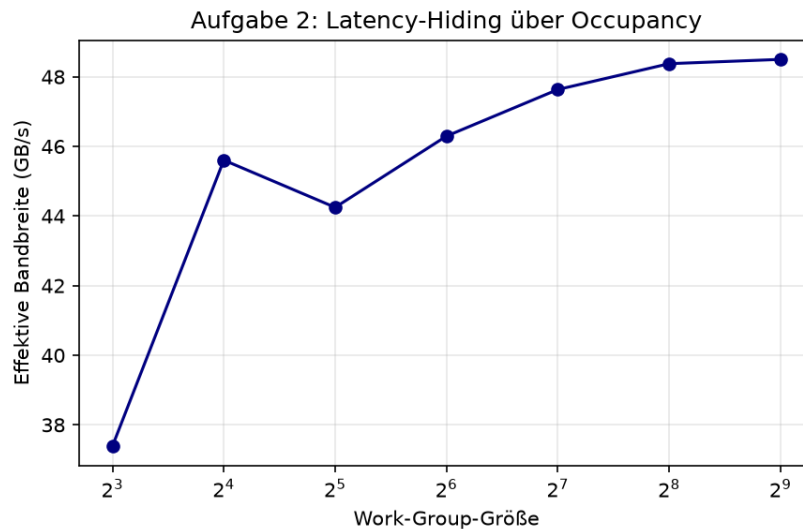


Abbildung 4: Bandbreite über der Work-Group-Größe (Occupancy).

| Messung                           | OpenCL<br>(POCL-CPU) | NumPy (CPU)     |
|-----------------------------------|----------------------|-----------------|
| Spitzen-Durchsatz (GFLOP/s)       | 20,7                 | 12,0            |
| Divergenz-Einbruch (Grad 1 zu 64) | Faktor 1,4           | nahezu konstant |
| Bandbreite coalesced (GB/s)       | 46,3                 | 5,6             |
| Bandbreite gather (GB/s)          | 9,2                  | 1,1             |

Tabelle 1: Zusammenfassung der Messungen.

steht nur bei divergenzarmen Kontrollflüssen und bei regelmäßigen, coalesceten Datenlayouts. Wo eine CPU über Sprungvorhersage und Caches einzelne unregelmäßige Zugriffe abfedert, bestraft die GPU sie durch Serialisierung der Pfade und durch zerfallende Speichertransaktionen. Für GPU-Anwendungen folgt daraus unmittelbar, Datenstrukturen so anzuordnen, dass benachbarte Threads benachbarte Daten lesen, und Verzweigungen innerhalb einer Wavefront möglichst zu vermeiden.